
MIDTERM LAB EXAM

Introduction to Programming

CSE141

Instructor: Ahmed Akhtar

Date: 10th October, 2026

Duration: 2 hours

Total Marks: 90

INSTRUCTIONS

1. There are a total of 4 questions out of which you must **attempt any 3 questions** carrying **30 marks** each.
2. Submitting only the **.out** or **.exe** file for a question will result in **0 marks**.
3. You will be given 5 minutes at the end to submit the files on **LMS**.
4. Only **VS Code** should be open throughout the duration of the exam. Before the specified time, you are not allowed to open the **web browser**.
5. Not following the instructions will result in **negative marking**.

Do not turn this page until instructed.

Question 1 — ATM Simulator**[30 marks]**

Simulate an ATM with a starting balance of **5000** and a correct PIN of **1234**.

1. **Login:** The user gets 3 PIN attempts. After each wrong attempt print **Wrong PIN. Attempts left: X**. After the 3rd failure print **Card blocked.** and end the program.
2. **Menu** (repeated using **do-while**, handled with **switch**):
 - **1** — Check balance
 - **2** — Deposit: amount must be > 0
 - **3** — Withdraw: amount must be a multiple of 500, at most 20000 per transaction, and not more than the balance. Check in this order and print a separate error message for each rule.
 - **4** — Show the last transactions: keep only the 5 most recent successful transactions in an array. When a 6th arrives, the oldest is dropped. Print them oldest to newest.
 - **0** — Exit: print total deposited, total withdrawn, final balance
 - Any other choice prints **Invalid choice**.

Sample Run

(Text after a prompt is typed by the user; the menu is redisplayed each time but omitted below.)

```
Enter PIN: 1111
Wrong PIN. Attempts left: 2
Enter PIN: 1234
Login successful.
Choice: 1
Balance: 5000
Choice: 2
Enter amount: 3000
Deposited 3000. New balance: 8000
Choice: 3
Enter amount: 2300
Invalid amount. Must be a positive multiple of 500.
Choice: 3
Enter amount: 2500
Withdrawn 2500. New balance: 5500
Choice: 3
Enter amount: 9000
Insufficient balance.
Choice: 2
Enter amount: 1500
Deposited 1500. New balance: 7000
Choice: 3
Enter amount: 500
Withdrawn 500. New balance: 6500
Choice: 2
Enter amount: 2000
Deposited 2000. New balance: 8500
Choice: 3
Enter amount: 1000
Withdrawn 1000. New balance: 7500
Choice: 2
Enter amount: 500
Deposited 500. New balance: 8000
Choice: 4
Last transactions:
1. Withdraw 500
2. Deposit 2000
```

```
3. Withdraw 1000
4. Deposit 500
5. Deposit 1500
Choice: 0
Total deposited: 7000
Total withdrawn: 4000
Final balance: 8000
Goodbye!
```

Question 2 — Result Card from File**[30 marks]**

A file `input.txt` has one student per line:

Name Marks1 Marks2 Marks3

Marks are out of 100, the name has no spaces, and there are at most 50 students. Read until the end of the file and write `report.txt` containing, for every student:

- total marks
- percentage (total / 3, 2 decimals)
- Grade by percentage: $A \geq 90$, $B \geq 80$, $C \geq 70$, $D \geq 50$, F below 50
- Status: PASS only if every subject is ≥ 50 , otherwise FAIL

After the table, write: class average percentage, the topper (name and percentage), number passed and failed, and the average of each subject. If `input.txt` cannot be opened, print **Error: could not open input.txt** on screen and stop. On success, print **Report written to report.txt** on screen.

Sample `input.txt`

```
Ali 85 90 78
Sara 92 88 95
Usman 45 60 52
Hina 70 65 80
```

Sample `report.txt`

```
Name Total Percentage Grade Status
Ali 253 84.33 B PASS
Sara 275 91.67 A PASS
Usman 157 52.33 D FAIL
Hina 215 71.67 C PASS
Class average: 75.00
Topper: Sara (91.67)
Passed: 3 Failed: 1
Subject averages: 73.00 75.75 76.25
```

Question 3 — Image Blur Filter**[30 marks]**

In image processing, a **blur filter** replaces each pixel with the average of itself and its 8 neighbouring pixels (a 3×3 box). Write a C++ program `image_blur.cpp` that reads a grayscale image as a 2D grid of integers (0–255) and prints the blurred version.

For **edge and corner pixels**, only average the neighbours that actually exist. Do not wrap around the grid, and do not treat missing neighbours as 0. Use integer division, so the result is rounded down.

Input

- First line: two integers *rows* and *cols* ($3 \leq rows, cols \leq 10$).
- Next *rows* lines: *cols* integers each (0–255), entered row by row.

Output

Print the blurred image, one row per line, with each pixel separated by a single space.

Sample Test Cases**Sample 1**

```
// Input
3 3
10 20 30
40 50 60
70 80 90

// Output
30 35 40
45 50 55
60 65 70
```

Sample 2

```
// Input
4 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

// Output
3 3 4 5
5 6 7 7
9 10 11 11
11 12 12 13
```

Question 4 — Library Book Tracker**[30 marks]**

A small library tracks the books it lends out. Each book has an ID (integer) and a number of days it has been borrowed. Write a C++ program `library_tracker.cpp` that reads the borrowing records and produces a summary report.

Input

- First line: an integer n ($1 \leq n \leq 30$), the number of records.
- Next n lines: each line has two integers — a book ID and the number of days borrowed.

Output

Print the following, each on its own line:

1. **Total borrowed days:** the sum of all days across every record.
2. **Average days:** the average days per record, printed with 2 decimal places. If n is 0, print **No records.** and stop.
3. **Longest borrow:** the book ID with the highest days. If two books tie, pick the one that appeared first.
4. **Overdue books:** print the IDs of all books borrowed for more than 14 days, in the order they appeared. If none, print **None.**
5. **Frequency:** the number of times each distinct book ID appears, printed as **ID: count**, one per line, in ascending order of ID.

Sample Input

```
6
101 10
102 20
101 5
103 7
102 18
104 25
```

Sample Output

```
Total borrowed days: 85
Average days: 14.17
Longest borrow: 104
Overdue books: 102 102 104
Frequency:
101: 2
102: 2
103: 1
104: 1
```

Requirements

1. Use two static 1D arrays: `int id[30]` and `int days[30]`.
2. Use a loop to read the records.
3. Compute the total and average using a loop.
4. Find the longest borrow by tracking the maximum and its index.
5. Collect overdue books with a loop and an `if` condition.

6. Count frequencies with a nested loop that marks already-counted IDs (or by scanning distinct IDs in ascending order).

DO NOT USE

- Vectors, strings, recursion, or user-defined functions.
- Any library function beyond `<iostream>` and `<iomanip>`.
- Hardcoded arrays or outputs.

END OF EXAM

Make sure you have submitted .cpp files of all your attempted questions in the Lab Exam submission tab on LMS.
